

Technology Guides — Current LoRaWAN Deployment

These files are the **technology-by-technology operator manuals** for the commissioned LoRaWAN infrastructure. They are written so that an operator can start with no assumed knowledge of the stack, identify the correct host, run safe checks in order, understand the expected result, and know where to continue when a check fails.

Read this first

The documentation authority order is:

1. verified live runtime state;
2. [../server/cloud-production/00-current-server-continuation-checkpoint.md](#);
3. current code, migrations, tests, service units, Compose files, and configuration in this repository;
4. these technology guides and their linked component runbooks;
5. build logs and archives only as historical evidence.

Before changing these manuals, read [CURRENT-DOCUMENTATION-BASELINE.md](#). It freezes the current AS923 wording, Fabric Task 37 contract, evidence eligibility rules, writer boundary, and other facts that must not regress into obsolete documentation.

For how these Markdown sources become **21 dedicated, separately reviewable PDFs**, see [PDF-PUBLICATION.md](#). Update Markdown first; PDFs are generated publications, not alternate source files.

How to use every guide

Follow the guide from top to bottom. Do not skip directly to a restart or configuration edit.

1. Read **What this technology does** so you know what layer you are testing.
2. Read **Where it runs** and make sure your shell is on the correct machine.
3. Run **Safe health check** before changing anything.
4. Compare the output with **PASS means**. A process being running is not enough when the guide also requires a real protocol or data-path check.
5. If a check fails, use **Troubleshooting order** from the first failing layer outward. Do not change healthy downstream components to compensate for an upstream failure.
6. Before any mutation, take the backup or rollback step named in the guide.
7. Make one related change at a time, validate syntax/configuration when the technology supports it, then reload/restart only the affected service.
8. Run **Post-change verification**. Stop if it fails and either repair the change or use the documented rollback.
9. Use the linked deployment/recovery runbook for full rebuilds, migrations, destructive recovery, or intentionally disruptive HA tests.

Command conventions

- Gateway-01 commands use the OpenWrt/BusyBox shell and normally run as root.

- ULC-01, ULC-02, and ULC-03 commands use Ubuntu and normally run as opsadmin; use sudo only where shown or required.
- Never paste a placeholder such as <PASSWORD>, <TOKEN>, <PRIVATE_KEY>, <DOMAIN>, or <HOST> literally into a production command. Resolve it from the current protected configuration first.
- Never print private-key bodies, passwords, OpenBao recovery material, Fabric enrollment secrets, OTAA root keys, or unrestricted DSNs into terminals that are being recorded.
- Read-only checks come before state-changing commands.
- Do not run a failover, reboot, package upgrade, database restore, queue purge, or destructive reset merely to prove a documentation edit.

Current technology map

#	Technology	Current role	Manual
01	Raspberry Pi / ChirpStack Gateway OS / OpenWrt	Edge gateway operating platform	01-gateway-os-openwrt.md
02	RAK5146 + ChirpStack Concentrator	AS923 LoRa RF concentrator and packet interface	02-rak5146-concentrator-as923.md
03	SIM7600 LTE	Production-primary Gateway-01 backhaul	03-sim7600-lte-backhaul.md
04	Mosquitto MQTT	Edge buffering plus cloud gateway/application messaging	04-mosquitto-mqtt.md
05	ChirpStack	LoRaWAN network/application server	05-chirpstack.md
06	etcd	Distributed consensus/DCS for HA coordination	06-etcd.md
07	PostgreSQL + Patroni + TimescaleDB	Durable HA application/evidence database	07-postgresql-patroni-timescaledb.md
08	HAProxy + PgBouncer	Service routing and PostgreSQL pooling	08-haproxy-pgbouncer.md
09	Valkey + Sentinel	ChirpStack runtime state/cache HA	09-valkey-sentinel.md
10	Node-RED	MQTT application processing, telemetry writer, outbox creator	10-node-red.md
11	Grafana	Read-only operational/research visualization	11-grafana.md
12	OpenBao	HA KMS/transit signing and protected secret operations	12-openbao.md
13	SeaweedFS	Raw/immutable evidence object storage	13-seaweedfs.md
14	Gateway evidence services	Journal ingest, MQTT witness, verification, trusted decoding	14-gateway-evidence-services.md
15	Hyperledger Fabric adapter	Source-bound HRC blockchain anchoring	15-hyperledger-fabric-adapter.md

#	Technology	Current role	Manual
16	Ubuntu + Docker Compose + DigitalOcean/VPC	Host/runtime/cloud substrate	16-runtime-and-cloud-platform.md
17	PKI / TLS / service identity	Transport authentication and workload identity	17-pki-tls-and-service-identity.md
18	RAK4631 / WisBlock Arduino sensor firmware	EMU-01 agriculture-node firmware, payload-v2, OTAA and counted/production profiles	18-rak4631-wisblock-sensor-firmware.md
19	Python research recorder / test automation	Formal evidence capture, test gating, sealing and measurement automation	19-research-recorder-and-test-automation.md
20	RUI3 security node	SEC RAK4631 authorized security-fixture firmware and safe parked state	20-rui3-security-node.md
21	End-to-end integration and data lineage	Cross-system contract, traceability, failure isolation, and research evidence chain	21-end-to-end-integration-and-data-lineage.md

Prometheus, Alertmanager, and Loki are **not** required parts of the commissioned small POC monitoring stack. The current observability model uses service-native health checks/logs, SQL evidence, the research recorder, and Grafana. Do not add a new monitoring platform merely because it appears in an old plan or generic architecture example.

Current end-to-end flow

EMU-01

- > AS923 LoRa RF
- > RAK5146 + Concentrator on Gateway-01
- > MQTT Forwarder
- > local Mosquitto persistent buffer
- > SIM7600 LTE + mTLS
- > cloud Mosquitto preferred/backup service
- > ChirpStack
- > Node-RED
- > PostgreSQL/TimescaleDB
- > Grafana
- > telemetry.fabric_outbox
- > Fabric Adapter on ULC-01
- > OpenBao seal/verify
- > HRC Hyperledger Fabric

Independent evidence lane:

Gateway journal + MQTT witness + application telemetry

- > evidence ingest/collectors/verifiers
- > trusted decoder
- > PostgreSQL metadata + SeaweedFS raw objects

Fast fault-location rule

When an end-to-end result is missing, start at the earliest layer that should have produced evidence and move forward only after it passes:

- ```
sensor serial -> RF/Concentrator -> local MQTT -> LTE/cloud MQTT
-> ChirpStack event -> Node-RED -> PostgreSQL -> Grafana
```

```
-> evidence_eligibility/finalized_payload -> Fabric_adapter -> HRC_ledger
```

This prevents a common mistake: restarting Fabric because an outbox row was never eligible, or changing ChirpStack because the gateway never received RF.

## Full rebuild order

---

For a new environment or disaster recovery, use the detailed runbooks linked by each guide. The normal dependency order is:

```
host/network/PKI
-> etcd
-> PostgreSQL/Patroni/TimescaleDB
-> HAProxy/PgBouncer
-> Valkey/Sentinel + Mosquitto
-> ChirpStack
-> OpenBao + evidence/object storage
-> Node-RED + Grafana
-> gateway OS/radio/local MQTT/LTE
-> evidence services
-> Fabric adapter
-> end-to-end verification
```

Do not interpret this rebuild order as permission to recommission healthy production layers.

---

Canonical source: [deployment/technology-guides/00-README.md](#). For any mutable production-status statement, check the dated baseline and verified live state.